

Nama : Imron
NPM : 232310021
Mata Kuliah : LAB PPB TI24PA2

Latihan 1

Latihan ini mengimplementasikan fitur pendaftaran akun baru pada aplikasi mobile berbasis React Native dan Expo. Proses ini menggunakan fungsi asinkron ADD_USER yang memanfaatkan pustaka Axios untuk mengirimkan payload berupa username, email, dan password ke server Fake Store API menggunakan metode HTTP POST. Jika server berhasil memproses data dan mengembalikan ID pengguna baru, data tersebut akan digabungkan lalu disimpan secara permanen di penyimpanan lokal perangkat menggunakan AsyncStorage dalam bentuk string JSON.

Pada sisi antarmuka halaman Signup, aplikasi melakukan validasi input terlebih dahulu untuk memastikan seluruh formulir telah terisi, format email yang dimasukkan sudah benar, serta kata sandi dan konfirmasinya sudah sinkron. Selama proses pengiriman data berlangsung, status pemuatan aktif untuk menonaktifkan tombol pendaftaran guna mencegah pengiriman ganda. Jika seluruh proses pembuatan akun dan penyimpanan lokal berhasil, aplikasi menampilkan notifikasi sukses dan secara otomatis mengalihkan navigasi pengguna ke halaman masuk atau Sign In.

A screenshot of a code editor with a dark blue background and light blue text. The code is for a function named ADD_USER, which is an asynchronous function that takes a parameter 'param'. It uses the Axios library to make a POST request to the Fake Store API. The request configuration includes the method 'POST', headers for 'Content-Type' as 'application/json', the URL 'https://fakestoreapi.com/users', and the data 'param'. The function returns the response from the API. If there is an error, it logs a warning and returns an object with 'loading' set to false and a message indicating an unexpected error during registration.

```
API.jsx

const ADD_USER = async (param) => {
  try {
    const config = {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
      },
      url: "https://fakestoreapi.com/users",
      data: param,
    };

    const response = await AXIOS_REQUEST(config);
    console.log("ADD_USER response:", response);
    return response;
  } catch (error) {
    console.warn("Error in ADD_USER:", error);
    return {
      loading: false,
      message:
        "An unexpected error occurred during registration. Please try again.",
      data: [],
    };
  }
};
```

```

import AsyncStorage from "react-native-async-storage/async-storage";
import { useRouter } from "expo-router";
import { useState } from "react";
import {
  Alert,
  KeyboardAvoidingView,
  Platform,
  ScrollView,
  StatusBar,
  Text,
  TouchableOpacity,
  View,
} from "react-native";
import { ADD_USER } from "../API";
import { Header } from "../components";
import { Buttons, InputText, TextPassword } from "../formUI";
import { style_auth } from "../styles";

const isValidEmail = (value) => {
  return /^[^\s@]+@[^\s]+\.[^\s]+$/.test(value);
};

export default function Signup() {
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [retypePassword, setReTypePassword] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const router = useRouter();

  const handleRegister = async () => {
    const trimmedUsername = username.trim();
    const trimmedEmail = email.trim();

    if (!trimmedUsername || !trimmedEmail || !password || !retypePassword) {
      Alert.alert("Error", "Semua field wajib diisi");
      return;
    }

    if (!isValidEmail(trimmedEmail)) {
      Alert.alert("Error", "Format email tidak valid");
      return;
    }

    if (password !== retypePassword) {
      Alert.alert("Error", "Password dan re-type password tidak sama");
      return;
    }

    setIsLoading(true);

    try {
      const payload = {
        email: trimmedEmail,
        username: trimmedUsername,
        password,
      };

      const results = await ADD_USER(payload);
      console.log("Register response:", results);

      if (results.message || !results.data || !results.data.id) {
        Alert.alert("Error", "Gagal membuat akun");
        return;
      }

      await AsyncStorage.setItem(
        "registeraduser",
        JSON.stringify({ ...payload, id: results.data.id })
      );

      Alert.alert("Success", "Akun berhasil dibuat", [
        {
          text: "OK",
          onPress: () => {
            router.replace("/module-latihan/latihan8/sign-in");
          },
        },
      ]);
    } catch (error) {
      console.error("Register error:", error);
      Alert.alert("Error", "Gagal membuat akun");
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <
      <StatusBar style="auto" backgroundColor="dark-content" hidden={false} />
      <KeyboardAvoidingView
        behavior={Platform.OS === "ios" ? "padding" : "height"}
        style={style_auth.container}
      >
        <ScrollView
          contentContainerStyle={style_auth.scrollContainer}
          keyboardShouldPersistTaps="handled"
        >
          <View style={style_auth.content}>
            <Header title="Sign up" />
            <View style={style_auth.form}>
              <InputText
                data={username}
                setData={setUsername}
                placeholder="Username"
              />
              <InputText
                data={email}
                setData={setEmail}
                ios="Mail outline"
                placeholder="Email"
                keyboardType="email address"
              />
              <TextPassword
                passwords={password} setPassword={setPassword} />
              <TextPassword
                passwords={retypePassword}
                setPassword={setReTypePassword}
                placeholder="Re-type Password"
              />
              <Buttons
                style={
                  style_auth.signInButton,
                  { loading: !style_auth.buttonDisabled },
                }
                onPress={handleRegister}
                disabled={!isLoading}
              >
                <Text style={style_auth.signInButtonText}>
                  {isLoading ? "Registering..." : "Register"}
                </Text>
              </Buttons>
            </View>
            <View style={style_auth.signUpContainer}>
              <Text style={style_auth.signUpText}>
                Already have an account?
              </Text>
              <TouchableOpacity
                onPress={() =>
                  router.replace("/module-latihan/latihan8/sign-in")
                }
              >
                <Text style={style_auth.signInLink}>Sign In</Text>
              </TouchableOpacity>
            </View>
          </ScrollView>
        </KeyboardAvoidingView>
      </
    >
  );
}

```

Latihan 2

Latihan ini mengimplementasikan mekanisme pembatasan akses atau proteksi rute pada halaman detail buku untuk memastikan keamanan data aplikasi. Alur kerja sistem dimulai saat komponen Detail dimuat, di mana fungsi asinkron `checkLoginStatus` langsung dipanggil melalui `useEffect` untuk memeriksa keberadaan data sesi pengguna seperti `authToken`, `userData`, atau bendera `isLoggedIn` di dalam `AsyncStorage`. Jika data tersebut tidak ditemukan atau bernilai kosong, aplikasi akan memicu kemunculan jendela peringatan sistem berisi pesan informatif bahwa pengguna wajib masuk terlebih dahulu, kemudian secara otomatis mengalihkan navigasi ke halaman Sign In menggunakan Expo Router dan memblokir render antarmuka detail buku dengan mengembalikan nilai kosong.

Selain proteksi di tingkat halaman, mekanisme pemeriksaan keamanan ini juga diterapkan secara berlapis pada fungsi penanganan tombol aksi bernama `handleReadBook`. Ketika pengguna menekan tombol untuk membaca atau berlangganan, fungsi ini kembali memvalidasi status login pengguna melalui penyimpanan lokal untuk memastikan sesi masih berlaku. Jika pengguna terverifikasi telah masuk, sistem akan memeriksa properti status buku untuk melakukan navigasi bersyarat; rute akan diarahkan ke halaman membaca jika buku tersebut bersifat gratis, atau dialihkan ke halaman langganan jika buku memerlukan akses berbayar. Melalui kombinasi pengecekan otomatis saat halaman dimuat dan validasi dinamis pada tombol aksi, aplikasi berhasil menerapkan kontrol akses berbasis autentikasi lokal yang adaptif dan aman.

Latihan 3

Fitur pemutusan sesi pengguna atau Sign Out dikelola oleh fungsi `handleSignOut` yang memanfaatkan jendela konfirmasi bawaan sistem `Alert.alert`. Dialog konfirmasi ini bertindak sebagai pengaman untuk mencegah tindakan keluar aplikasi yang tidak disengaja dengan menyediakan pilihan bagi pengguna untuk membatalkan proses atau melanjutkan eksekusi.

Ketika pengguna menyetujui proses keluar, fungsi asinkron `logout` akan dijalankan untuk membersihkan seluruh data kredensial yang tersimpan. Proses pembersihan ini menggunakan metode `AsyncStorage.multiRemove` guna menghapus kunci `authToken`, `userData`, dan status `isLoggedIn` sekaligus dalam satu waktu. Setelah seluruh data sesi berhasil dihapus dari penyimpanan lokal untuk menjaga keamanan akun, navigasi akan langsung dialihkan kembali menuju rute halaman login utama menggunakan `Expo Router`.

[illegible]

Output

